

Chicago TNP Full-Data Analytics and Forecasting Project

H3 Demand, Tip Prediction, LLM Tests, Model Deployment, and Chronos-2

Eric / Junjiang Chen 2026-07-20

Executive summary. This project built a full-data workflow for 2022-2024 Chicago Transportation Network Provider trips. We stored and aggregated 243,479,296 completed trips in MatrixOne, converted locations to H3 cells, and tested several prediction tasks. The strongest standard model predicted next-hour H3 pickups with MAE 12.50 and R^2 0.945. The two-stage tip model reduced expected-tip MAE from \$1.576 to about \$1.415, but tip/no-tip prediction remained difficult. LLM tests were useful as feasibility studies, but they did not beat the best tree or time-series models. In the last experiment, the pretrained Chronos-2 history-only model gave the best overall hourly forecast MAE of 30.41 across six horizons and ten busy H3 cells.

1 Project Background

The original goal was to study price and demand patterns in a very large ride-hailing dataset. As the work continued, the project became broader. The main questions were:

- Can we predict short-term demand for one H3 area?
- Can pickup, dropoff, and nearby H3 activity help us detect peaks and event-like patterns?
- Can time and location predict whether a trip has a recorded tip and how much the expected tip is?
- Can an LLM find the same patterns as traditional models?
- Can a Python model be exported and used in Go without changing its prediction?
- Can a pretrained time-series model such as Chronos-2 beat LightGBM?

The project is not only about model accuracy. It also tests a practical workflow: MatrixOne handles large SQL aggregation, Python handles modeling, ONNX supports model exchange, Go tests deployment, and LLM APIs test reasoning on small structured summaries.

2 Data Source and Preparation

The source is the City of Chicago public Transportation Network Provider trip data for 2022-2024. Two full CSV files were cleaned and merged: `tnp_2022_full.csv` and `tnp_2023_2024_full.csv`. The final MatrixOne H3 table, `chicago_tnp.unified_trips_h3_res9`, contains 243,479,296 rows: 69,109,780 from 2022 and 174,369,516 from 2023-2024.

The main fields are trip start and end time, duration, miles, fare, recorded tip, pickup and dropoff areas, centroid coordinates, and pickup/dropoff H3 IDs. H3 resolution 9 turns a location into a stable area code, so we can group trips by place and time and also study first-ring neighbor cells.

There are two important limits. First, this is completed-trip data. It does not include all ride requests, failed matching, driver supply, waiting time, or cancellations. Second, the `tip` field is the recorded tip. Cash tips may not appear. For this reason, the report uses the terms *completed pickup demand* and *recorded tip*, rather than claiming full market demand or full customer tipping behavior.

3 Main Tests in Time Order

3.1 Hourly H3 demand, sensitivity, flow, and spatial spread

We first aggregated the full table into hourly pickup, hourly dropoff, and first-ring neighbor pickup counts for the top 50 H3 cells selected from 2022-2023. We trained LightGBM models to predict demand 1, 2, 3, and 6 hours ahead. Features included normal weekday-hour demand, recent pickup lags, rolling averages, z-score changes, dropoff flow, and neighbor activity. The split was time based: 2022-2023 for training and 2024 for testing.

The base model reached MAE 13.01 and R^2 0.939 at one hour. The MAE values for 1, 2, 3, and 6 hours were 13.01, 15.33, 16.29, and 16.93. Adding pickup/dropoff flow and spatial features improved one-hour MAE to 12.50 and R^2 to 0.945. A sensitivity test also showed that the model reacts when the current hour falls: when current pickups were reduced to 70% of their original value, the average next-hour prediction fell by 15.2%; at 30%, it fell by 37.6%. The response was real but smoother than the input drop.

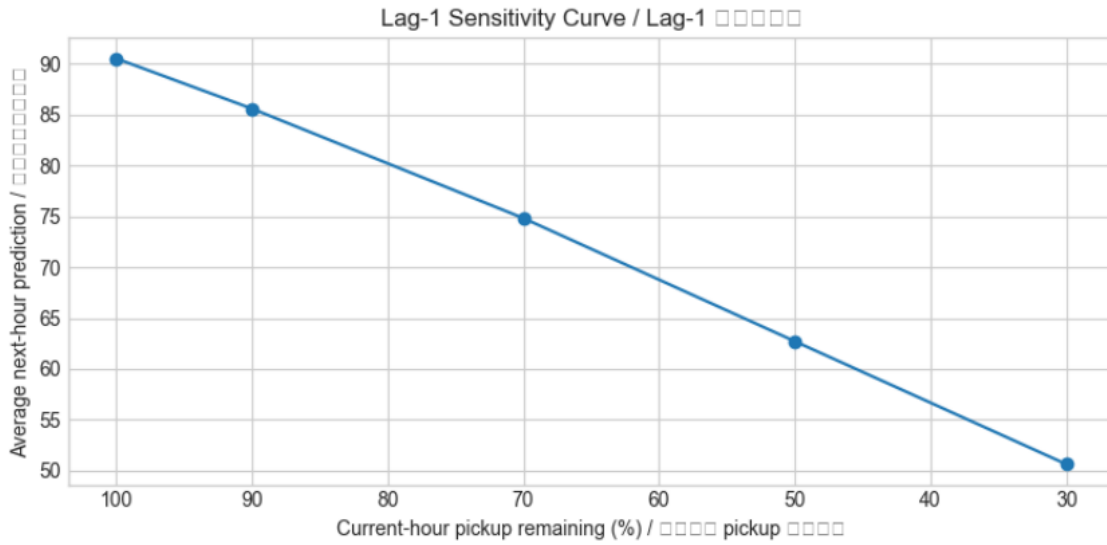


Figure 1: Lag-1 sensitivity and next-hour feature importance.

This figure shows how the next-hour forecast changes when the current-hour pickup count is reduced. The horizontal axis is the percentage of current pickups kept, and the vertical axis is the model's average next-hour prediction. Moving from 100% toward 30% simulates a sudden demand drop. The downward curve confirms that the model notices the change, but it does not copy it one for one. This smoothing reduces noise, although it may underestimate a very sudden collapse. Feature importance shows which variables often help the model, but it describes association rather than cause.

We also removed normal weekday-hour patterns and compared dropoff and later pickup z-scores. The strongest relation was in the same hour: 0.427 in 2022-2023 and 0.449 in 2024. The correlation became weaker as the dropoff lead increased. This means dropoff information is useful, but there was no strong general rule that a dropoff peak always creates a pickup peak several hours later. The notebook also produced 883 regional event-center candidates. Center-to-neighbor correlation was positive but small and decreased from 0.131 at the same hour to 0.050 after three hours, so these are candidates for review, not proof of event spread.

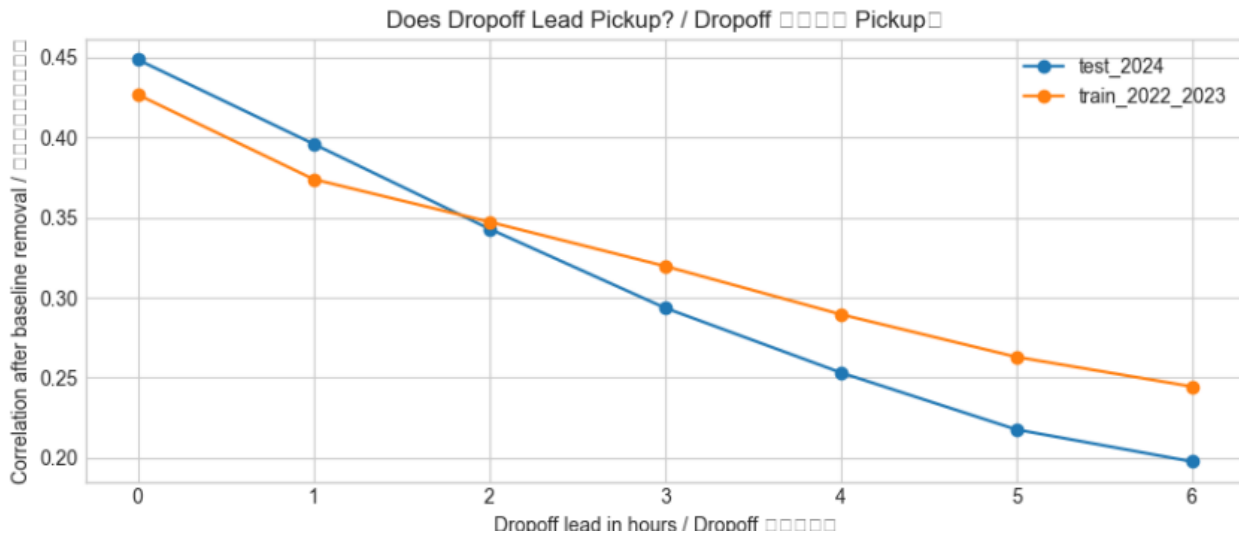


Figure 2: Dropoff-to-pickup cross-lag relationship after removing the normal baseline.

This figure shows the relationship between current dropoffs and pickups in the same or later hours after normal weekday-hour patterns are removed. Zero on the horizontal axis compares the same hour, while two compares dropoffs now with pickups two hours later. The line is highest at zero and then declines, meaning pickup and dropoff pressure

often happen together. However, the data do not support one fixed city-wide delay from a dropoff peak to a later pickup peak, and individual H3 areas may follow different patterns.

3.2 Two-stage recorded-tip prediction

The next task predicted the recorded tip using pickup time, dropoff time, and location changes. Tip data contain many zeros, so one regression model is not a good fit. We used two LightGBM stages:

$$\text{Expected tip} = P(\text{recorded tip} > 0) \times E(\text{tip amount} \mid \text{tip} > 0).$$

Stage 1 is a classifier for tip versus no tip. Stage 2 is a regressor trained only on positive-tip trips. We tested three feature groups: pickup-time information, time plus location change, and completed-trip information including fare, miles, seconds, and speed. `trip_total` was excluded because it may already include the tip and would leak the answer. The yearly recorded-tip rate was close to 25-26%, and the average positive recorded tip was about \$4.84-\$5.04. The training-mean baseline had expected-tip MAE \$1.576. The three models gave very similar MAE: \$1.415 for pickup-time information, \$1.418 for time plus location change, and \$1.416 for completed-trip information. The completed-trip classifier had ROC-AUC 0.655. Therefore, the model learned a real ranking signal, but time and location alone were not enough for a highly accurate tip decision. The decile chart was more encouraging: higher predicted groups also had higher actual average tips.

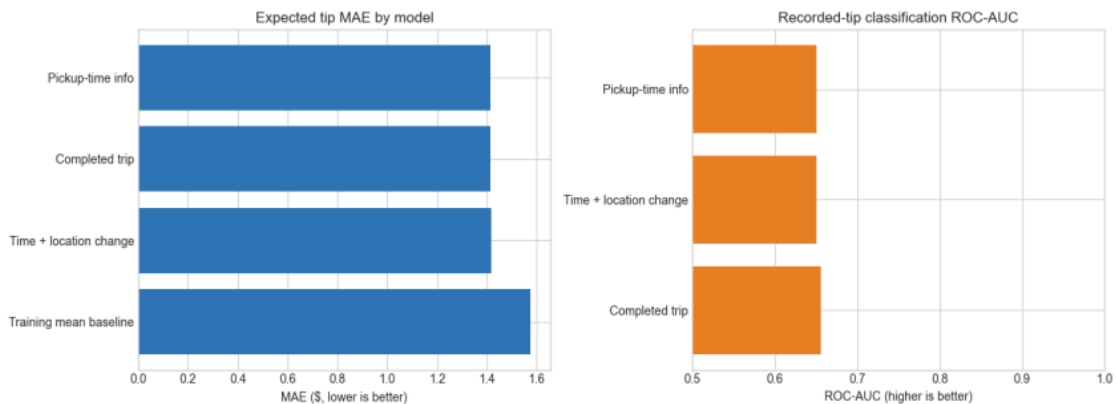


Figure 3: Two-stage tip model comparison and predicted-versus-actual deciles.

This figure shows the error of each two-stage tip model and how predicted tip groups compare with actual tips. Shorter MAE bars are better. The prediction groups run from the lowest predicted tips to the highest, so an increasing actual-tip pattern means the model can rank trips even when the exact dollar estimate is imperfect. All three feature sets beat the average baseline, but their MAE values are nearly the same. The model is therefore more useful for separating lower- and higher-tip trips than for predicting one exact tip, probably because payment method, customer history, service quality, and unrecorded cash tips are missing.

3.3 LLM peak-continuation forecast

We then tested a narrow LLM question: after Python has already confirmed a current peak, will the same H3 cell still be in a peak 1, 3, or 6 hours later, and what will the pickup count be? Python created compact structured cases containing current pickup, expected pickup, z-score, recent trend, lag values, dropoff pressure, neighbor pressure, and evidence IDs. GPT-5-mini received these summaries, not the 243 million raw rows.

This was a smoke test with 60 cases, 20 for each horizon, balanced between continued and ended peaks. At one hour, the LLM had MAE 67.4 and F1 0.667, close to the current-hour baseline with MAE 63.9 and F1 0.692. At three hours, GBM was clearly better: MAE 34.4 and F1 0.750 versus LLM MAE 92.0 and F1 0.615. At six hours, GBM also had lower MAE, 57.3 versus 140.6. LLM latency was about 9.8-14.0 seconds per case. The test showed that LLM access and structured reasoning worked, but it did not show better numeric forecasting.

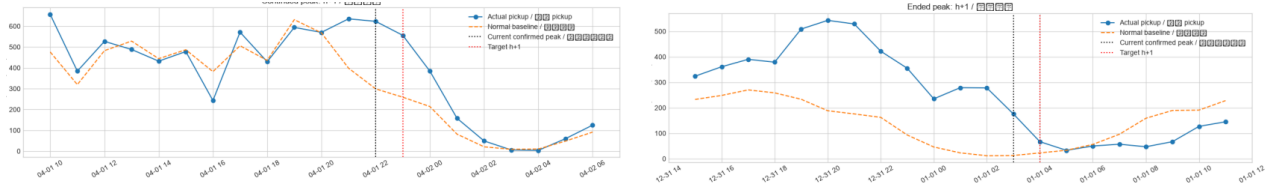


Figure 4: Examples of a continued peak and an ended peak.

This figure shows one peak that continues and one that ends. The solid pickup line is actual completed pickups, the dashed line is the normal weekday-hour baseline, the black vertical line is the time when the current peak is confirmed, and the red line is the future target time. The black line is not selected as the highest point in the chart. A future point is called a continued peak only when it remains above that future hour’s own threshold, so a later value can be higher than the black-line value but still fail the rule if its normal baseline is also higher. This is why the forecast needs the recent trend, peak age, and future-hour baseline.

3.4 Python-to-ONNX-to-Go portability

The next-hour H3 model was exported from Python to ONNX and loaded in a separate Go session. The same feature order and preprocessing values were used in both environments. The local test produced matching Python and Go predictions. This result is important because it shows that the model is not limited to a notebook. It can be moved toward a service written in Go, as long as feature names, order, types, and preprocessing are versioned together with the ONNX file.

3.5 LLM versus Tree for recorded-tip yes/no

The two-stage tip notebook was extended with a direct yes/no comparison on the same 200 rows from 2024. The sample kept the natural class ratio: 52 rows had a recorded tip and 148 did not. We compared a majority baseline, the existing LightGBM classifier, a depth-4 explanation tree, and GPT-5.4-nano. The LLM prompt used time, H3 locations, route frequency, fare, miles, duration, and speed, but never included the true tip label.

The explanation tree is not the full LightGBM model. It is a small tree trained only to show readable rules. It starts with `log_fare`. The only path that reached a “has tip” leaf required fare above about \$23.73, a route seen more than about 31 times, duration above about 26.2 minutes, and one side of a coded dropoff-H3 split. Even there, the leaf was only 50.3% positive. H3 codes are category IDs, so a rule such as `dropoff_h3_code > 39.5` is a program split, not a geographic statement such as east versus west.

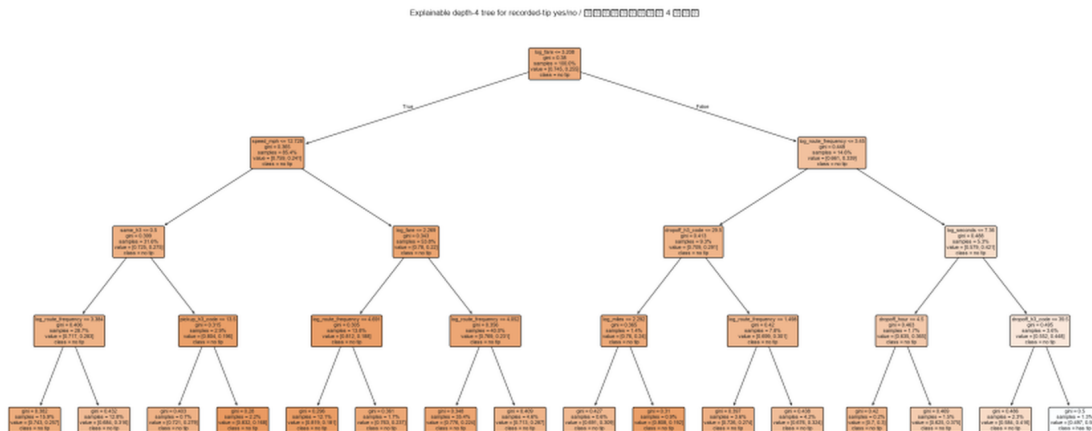


Figure 5: Depth-4 explanation tree for recorded-tip presence.

This figure shows the depth-4 decision tree used to explain the recorded-tip yes/no logic. Reading starts at the top box:

a true condition follows the left branch and a false condition follows the right branch until a leaf is reached. **samples** is the share of training data reaching a node, **value** is the no-tip and has-tip mix, **class** is the final decision, and a Gini value near zero means a cleaner group. The tree mostly predicts no tip, while its only positive path is narrow and only slightly above 50% positive. This suggests that the available fields do not form a strong and simple tipping rule.

On the 200 common rows, the shallow tree had 75.5% accuracy but only 5.8% recall for positive tips. LightGBM had 74.0% accuracy and 3.9% recall. GPT-5.4-nano had lower accuracy, 52.5%, but much higher recall, 51.9%, with F1 0.362. Balanced accuracy for all learned methods was only about 51-53%. The tree looked accurate because 74% of rows had no tip, while the LLM predicted “yes” too often and created 70 false positives. Neither approach was ready for use at the default threshold.

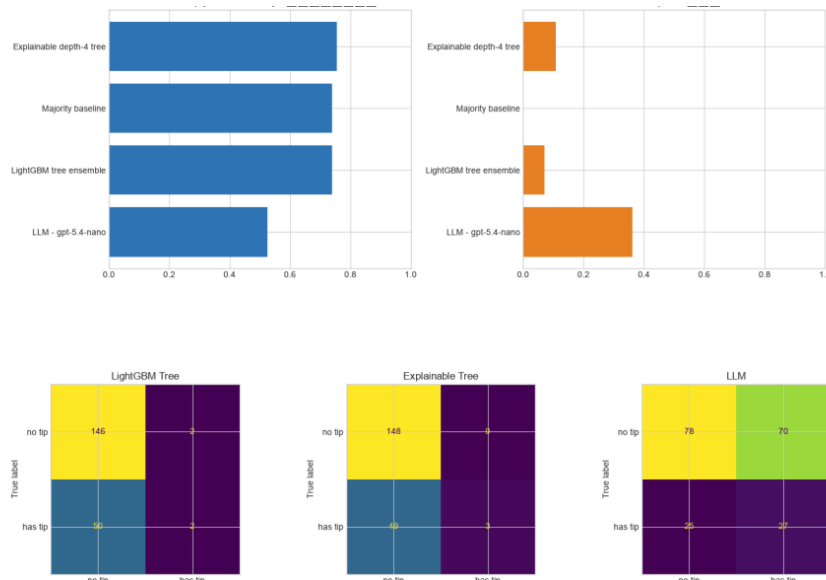


Figure 6: Accuracy, positive-tip F1, and confusion matrices for LLM and tree methods.

This figure shows accuracy, positive-tip F1, and confusion matrices for the tree and LLM methods. Accuracy counts every correct answer, while F1 gives more attention to the smaller positive-tip group. Diagonal cells in each matrix are correct decisions, the upper-right cell is a false positive, and the lower-left cell is a missed real tip. The tree keeps a higher overall accuracy mainly by answering “no tip” most of the time. The LLM finds more real positive tips but also creates many false alarms, showing why accuracy alone is misleading for this unbalanced target.

3.6 Chronos-2 versus LightGBM

The final test compared the pretrained `autogluon/chronos-2-small` model with time-aligned LightGBM and a last-week seasonal baseline. It used the top 10 H3 cells, 121 forecast origins in 2024, and horizons of 1, 2, 3, 6, 12, and 24 hours. This created 29,040 common predictions. Chronos ran in an isolated Python 3.11 worker so a model-library problem would not stop the notebook. LightGBM used leakage-safe lag features and exactly the same target timestamps.

Chronos-2 with history only was best overall: MAE 30.41, RMSE 52.53, and R^2 0.899. LightGBM with calendar features was close, with MAE 31.00 and R^2 0.887. The last-week baseline was much worse, with MAE 44.49. Chronos was strongest at short horizons, including one-hour MAE 23.22 versus 29.56 for LightGBM. Around 12-24 hours the two methods were close. Adding weather did not improve overall MAE: it increased Chronos MAE by 0.20 and LightGBM MAE by 0.35. Some weather slices improved, but the direction was not stable enough for a broad claim.

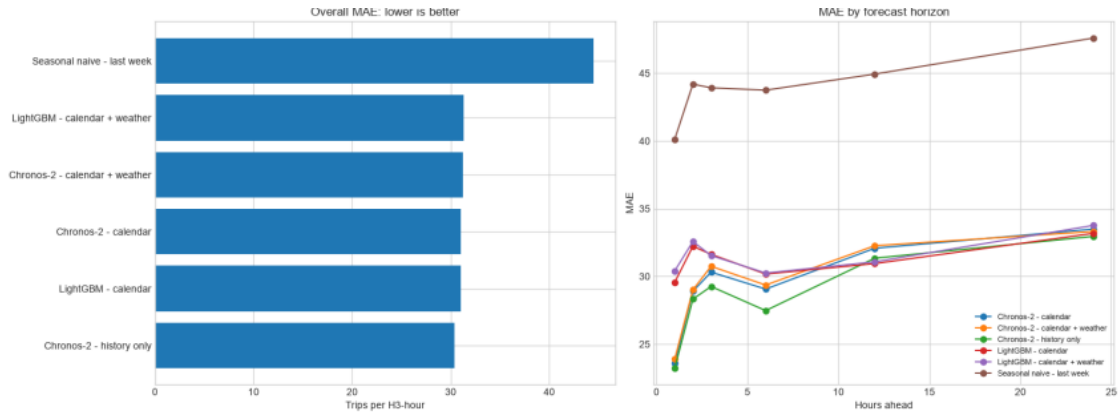


Figure 7: Overall MAE and MAE by forecast horizon.

This figure shows overall MAE and how forecast error changes from one to 24 hours ahead. Shorter bars and lower lines are better. Chronos history-only has the best average result and a clearly better one-hour forecast than the benchmark LightGBM, while the difference becomes smaller at longer horizons. Calendar and weather inputs do not automatically improve the result. The comparison suggests that a pretrained time-series model can be strong without task-specific training, although LightGBM remains easier to inspect, export, and deploy.

4 Result Summary

Test	Main result	Main meaning
Hourly H3 forecast	Full flow + spatial LightGBM: one-hour MAE 12.50, R^2 0.945	Recent pickups are strongest; dropoff flow helps; neighbors add a smaller gain.
Two-stage tip	Best expected-tip MAE about \$1.415 vs \$1.576 baseline; ROC-AUC 0.655	Useful ranking signal, but exact tipping remains hard.
LLM peak continuation	LLM close at h+1, clearly behind GBM at h+3 and h+6	LLM works on summaries, but is not the best numeric forecaster.
ONNX and Go	Python and Go predictions matched in local validation	The model can move outside the notebook.
LLM vs Tree tip/no-tip	Tree accuracy 75.5% with recall 5.8%; LLM accuracy 52.5% with recall 51.9%	Tree favors the majority class; LLM over-predicts positive tips.
Chronos-2 vs LightGBM	Chronos history-only overall MAE 30.41 vs LightGBM 31.00	Pretrained time-series modeling was strongest overall; weather was not consistently helpful.

5 Models and Artifacts Produced

- **Hourly H3 LightGBM models:** separate 1, 2, 3, and 6-hour models, plus flow, neighbor, and full feature versions.
- **Two-stage tip model:** a LightGBM classifier for tip presence and a LightGBM regressor for positive-tip amount.
- **Explainable tip tree:** a depth-4 decision tree used to show readable choices, not to replace the full LightGBM ensemble.
- **LLM evaluation pipelines:** GPT-5-mini for peak continuation and GPT-5.4-nano for tip/no-tip classification with structured JSON outputs.
- **ONNX next-hour model:** exported from Python and validated in Go with matching predictions.
- **Chronos-2 benchmark:** pretrained `chronos-2-small` history-only and covariate versions, compared on a shared backtest with LightGBM.

6 Planned Next Work

1. **Expand and calibrate the evaluation.** Use larger samples with natural class ratios, report bootstrap confidence intervals, PR-AUC, recall, latency, and API cost, tune tip-model thresholds, and extend the Chronos backtest to more H3 cells, forecast origins, and event-heavy periods.

2. **Build an event simulator with richer data.** After the first event day or hour is observed, hide the remaining observations and forecast later demand, peak duration, and spread to neighboring H3 cells. Add correctly aligned weather and event information, and use request, driver-supply, waiting-time, cancellation, or failed-matching data if they become available.
3. **Connect statistical models, LLM reasoning, and deployment.** MatrixOne should first turn the full trip table into compact weekly or event summaries. Compare LLM classification and explanation with tree and time-series baselines on the same cases, then package preprocessing, feature order, ONNX model versions, Go inference tests, latency checks, and a small command-line or HTTP demo together.